Beijing Air Quality Time-Series Pipeline — Group Report

Oct 29, 2026 · Kyla Ochweri, Nicole Mbera, David Shumbusho, Frank Musiime

Team and contributions

We built an end-to-end pipeline that forecasts next-hour PM2.5 in Beijing: a cleaned and explored dataset, two databases, a REST API over both, and a script that pulls a live record from the API and predicts with the trained model.

Member	GitHub	Task	Main contributions	Location in repository
Kyla Ochweri	`kylaochweri`	Task 1: Preprocessing, EDA and model	Dataset loading and timestamp construction, missing-value analysis, analytical questions and plots, Random Forest model, hyperparameter tuning, experiment table	`notebooks/`
Nicole Mbera	`NicoleMbera`	Task 2: MySQL and MongoDB	Three-table relational schema and ERD, MongoDB document design, data loading, five SQL and four MongoDB queries with results	`notebooks/`, `database/schema_mysql.sql`
David Shumbusho	`Shumbusho43`	Task 3: API development	FastAPI service with CRUD and time-series endpoints for both databases, data loader, hosted deployment, 26 automated API tests	`api/`, `database/load_data.py`, `tests/test_api.py`

Member	GitHub	Task	Main contributions	Location in repository
Frank Musiime	`frankmusiime`	Task 4: Prediction and integration	Prediction script (API to model), saved model pipeline, 5 prediction tests, README coordination	`prediction/predict.py`, `models/best_rf.joblib`, `tests/test_prediction.py`

- **GitHub repository:** github.com/Shumbusho43/Formative1MLPipeline
- **Live API (Swagger docs):** formative1mlpipeline.onrender.com/docs. It runs on Render's free tier, so the first request after about 15 idle minutes can take up to a minute.

1. Problem definition and dataset justification

Problem. Given the pollutant and weather readings at a monitoring station for the current hour, predict the PM2.5 concentration (µg/m³) for the next hour. This is a one-step-ahead time-series regression problem, modelled for the Dongsi station.

Why it matters. PM2.5 particles (⩽ 2.5 µm) reach deep into the lungs and are linked to cardiovascular and respiratory disease. Beijing's levels far exceed health guidelines: every one of the 12 stations averages 66–86 µg/m³, against a WHO 2021 annual guideline of 5 µg/m³. A reliable short-term forecast lets residents, schools and health services act before a pollution spike, for example by limiting outdoor activity or issuing alerts.

Dataset. We used the Beijing Multi-Site Air Quality dataset from Kaggle (UCI origin). It meets every requirement of the assignment:

Requirement	How the dataset meets it
Clear timestamp	`year`, `month`, `day`, `hour` columns, combined into one `timestamp`
Meaningful target	PM2.5 concentration, the pollutant most tied to health outcomes
Multiple variables over time	6 pollutants (PM2.5, PM10, SO2, NO2, CO, O3) and 6 weather variables (temperature, pressure, dew point, rain, wind direction, wind speed)
Enough history	4 years of hourly data, 420,768 rows across 12 stations

The dataset also suits the database tasks: its station-by-hour structure maps naturally to a normalised relational schema, and its grouped pollutant and weather values map to nested MongoDB documents. Real gaps in the readings gave us a genuine missing-data problem to handle.

2. Task 1A: Understanding the dataset

The data is a clean, regular hourly series: 12 stations × 35,064 hours each, with no duplicate station-hours and no gaps in the time index. Only the measurement values are missing, at 0.1–5% per column.

Time range and frequency. The 12 station CSVs were combined (420,768 rows) and a `timestamp` was built from `year`, `month`, `day` and `hour`. The series runs from 2013-03-01 00:00 to 2017-02-28 23:00. At every station, consecutive readings are exactly one hour apart, and there are 0 duplicate (station, timestamp) pairs.

Missing values. Every timestamp is present; gaps are only in the measured values:

Column	Missing rows	Missing (%)
CO	20,701	4.92
O3	13,277	3.16
NO2	12,116	2.88
SO2	9,021	2.14
PM2.5 (target)	8,739	2.08
PM10	6,449	1.53
Wind direction	1,822	0.43
TEMP, PRES, DEWP, RAIN, WSPM	318–403 each	⩽ 0.10

How we handled them, and why:

1. **Target gaps were dropped.** Rows whose next-hour PM2.5 was missing were removed before modelling (750 missing PM2.5 hours at Dongsi). Imputing the target would train the model on invented answers.
2. **Feature gaps were filled with the training-set median.** A `SimpleImputer(strategy="median")` is the first step of the model pipeline, so it learns medians from the training period only and applies them unchanged to the test period. This avoids leaking future information. We chose the median over the mean because the pollutant distributions are heavily right-skewed.

3. **Databases keep gaps as NULL.** The SQL and MongoDB stores keep missing readings as `NULL` / `null` rather than filled values, so the raw record stays faithful and each consumer can apply its own imputation.

Statistical distribution (all stations):

Variable	Mean	Median	Std	Min	Max
PM2.5 (µg/m³)	79.8	55.0	80.8	2.0	999.0
PM10 (µg/m³)	104.6	82.0	91.8	2.0	999.0
SO2 (µg/m³)	15.8	7.0	21.7	0.3	500.0
NO2 (µg/m³)	50.6	43.0	35.1	1.0	290.0
CO (µg/m³)	1,230.8	900.0	1,160.2	100.0	10,000.0
O3 (µg/m³)	57.4	45.0	56.7	0.2	1,071.0
Temperature (°C)	13.5	14.5	11.4	−19.9	41.6
Pressure (hPa)	1,010.7	1,010.4	10.5	982.4	1,042.8
Dew point (°C)	2.5	3.1	13.8	−43.4	29.1
Rain (mm)	0.1	0.0	0.8	0.0	72.5
Wind speed (m/s)	1.7	1.4	1.2	0.0	13.2

PM2.5 is strongly right-skewed (skewness 2.01): the mean (79.8) sits well above the median (55.0), the 99th percentile is 370 and the maximum is 999. Most hours are moderate, with rare, extreme pollution episodes. The other pollutants show the same pattern, while the weather variables are roughly symmetric. Station means for PM2.5 range from 66.0 (Dingling, suburban) to 86.2 (Dongsi, central).

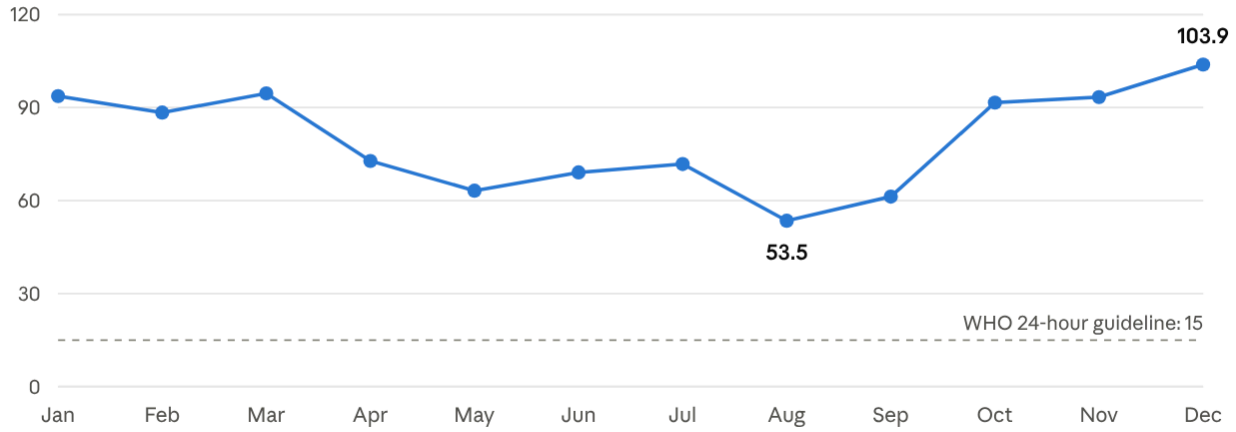
3. Task 1B: Analytical questions

We asked six questions. The strongest finding is persistence: this hour's PM2.5 almost fully predicts the next hour's (r = 0.97), but that link fades within a day. Questions 4 and 5 use lagged features and moving averages, as the assignment requires. Figures are computed from the raw data; lag and moving-average questions use the Dongsi station so that shifts never cross between stations.

Q1. Does PM2.5 have a trend or a seasonal pattern?

Winter PM2.5 runs about twice the summer level

Average PM2.5 by calendar month, all 12 stations, Mar 2013 to Feb 2017 (µg/m³)



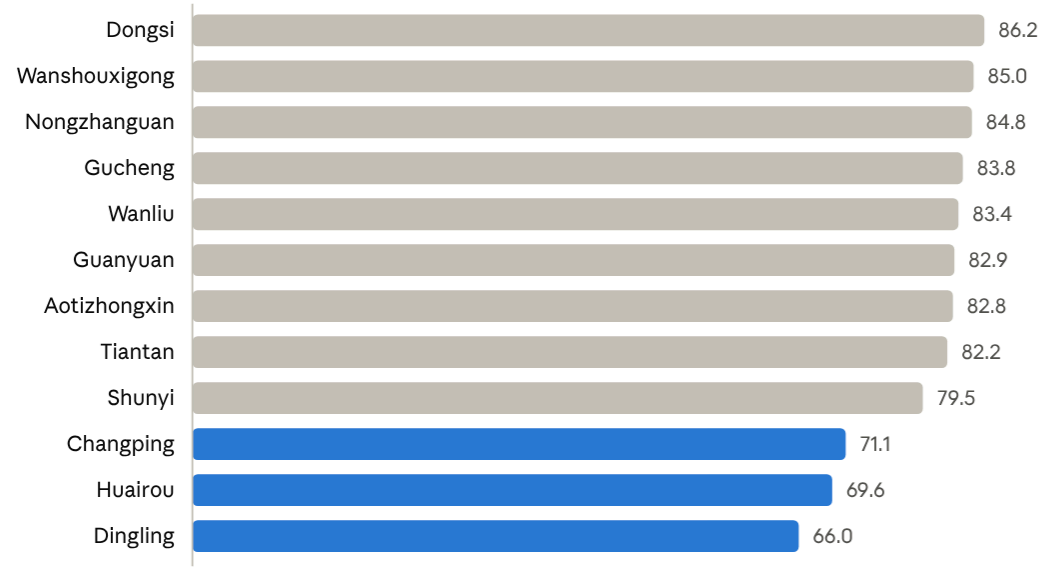
Computed from the 12 station CSVs · monthly means averaged by calendar month

Seasonality dominates. December averages 103.9 µg/m³ and August 53.5, almost a 2× swing, consistent with winter coal heating and temperature inversions that trap pollution near the ground. The long-term trend is mildly downward: averaged over March-to-February years, PM2.5 fell from 86.7 (2013–14) to 81.3 and 73.1, then rose to 78.1 in 2016–17. Four years is too short to call this a lasting decline, and even the cleanest month is over 3 times the WHO 24-hour guideline.

Q2. Which stations are the most polluted?

Three outlying stations average 66 to 71; the other nine 79 to 86

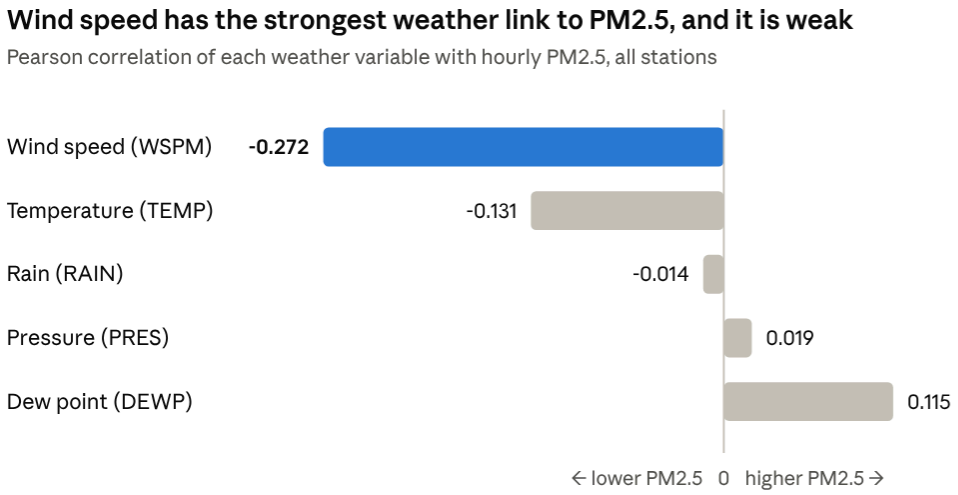
Mean PM2.5 per station, Mar 2013 to Feb 2017 (µg/m³). Highlighted: the three outlying northern stations.



Computed from the 12 station CSVs · same values as SQL Q3 and MongoDB MQ3

Nine stations cluster tightly between 79.5 and 86.2 µg/m³, led by central Dongsi. The three outlying northern stations (Changping, Huairou, Dingling) are 13–20 µg/m³ lower, consistent with less traffic and heating density away from the city. Even the cleanest station averages 13 times the WHO annual guideline (5 µg/m³). We modelled Dongsi because it is the most polluted station and the most important to forecast.

Q3. Do weather variables correlate with PM2.5?



Computed from the 12 station CSVs · pairwise-complete Pearson r over 420,768 hours

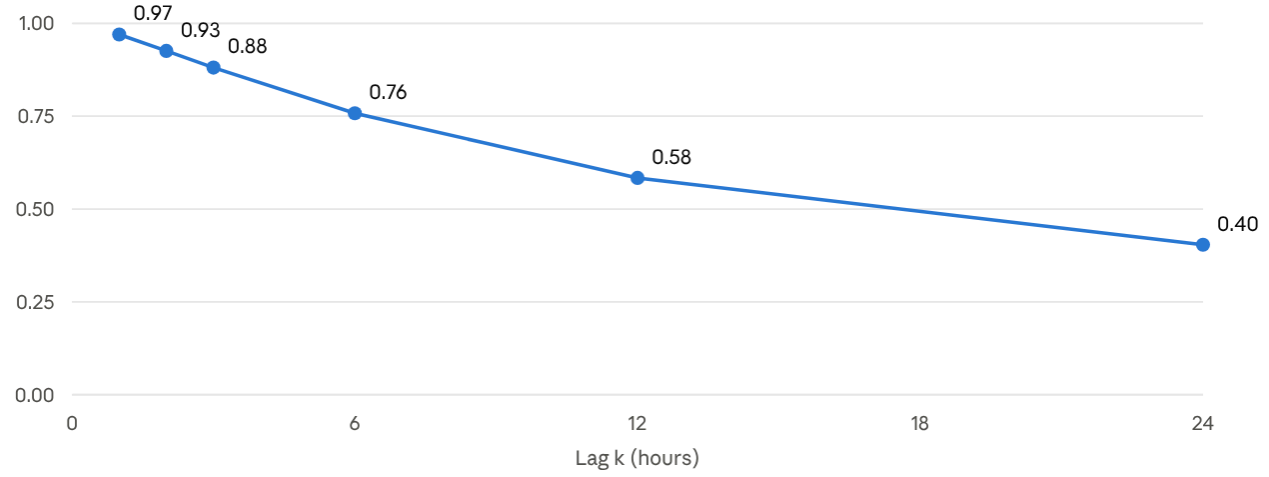
Only weakly, and wind matters most. Higher wind speed goes with lower PM2.5 (r = −0.27), as wind disperses particles. Warmer hours are slightly cleaner (r = −0.13), and humid hours slightly dirtier (dew point r = +0.12), consistent with stagnant, moist winter air. Rain and pressure show almost no linear link. No single weather variable explains much, which is why the model leans on the pollutant history instead (Q4).

Q4. Are there lag effects? (lagged features)

We built lagged features `PM2.5_lag_k = PM2.5.shift(k)` on the time-ordered Dongsi series and correlated each with the current value.

Last hour's PM2.5 predicts this hour almost perfectly; the link fades over a day

Correlation between PM2.5 now and PM2.5 k hours earlier (lag feature), Dongsi station



Computed from PRSA_Data_Dongsi · PM2.5 vs PM2.5.shift(k) for k = 1, 2, 3, 6, 12, 24

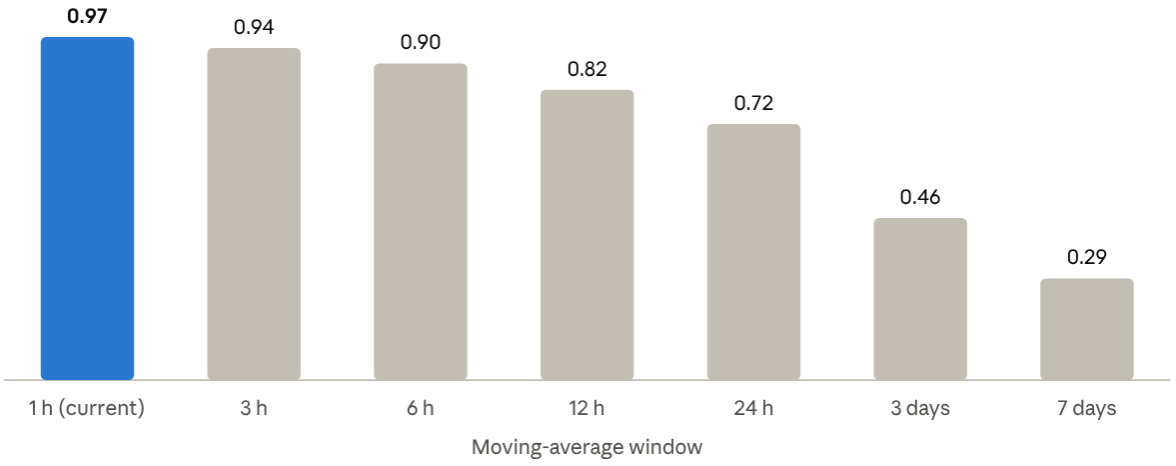
PM2.5 is highly persistent. The previous hour alone correlates at r = 0.97 with the current hour, so pollution episodes build and clear over many hours rather than switching on and off. The link decays steadily: 0.88 after 3 hours, 0.58 after 12 and 0.40 after 24. For one-hour-ahead forecasting, the most recent reading is by far the best predictor. This explains why current PM2.5 carries 97.7% of the model's feature importance.

Q5. Do moving averages help forecast the next hour? (moving averages)

We computed trailing moving averages `PM2.5.rolling(window).mean()` at Dongsi for windows from 3 hours to 7 days, then plotted the hourly series against its 24-hour average (January–February 2016) and correlated each average with the next hour's PM2.5.

The longer the moving average, the worse it forecasts the next hour

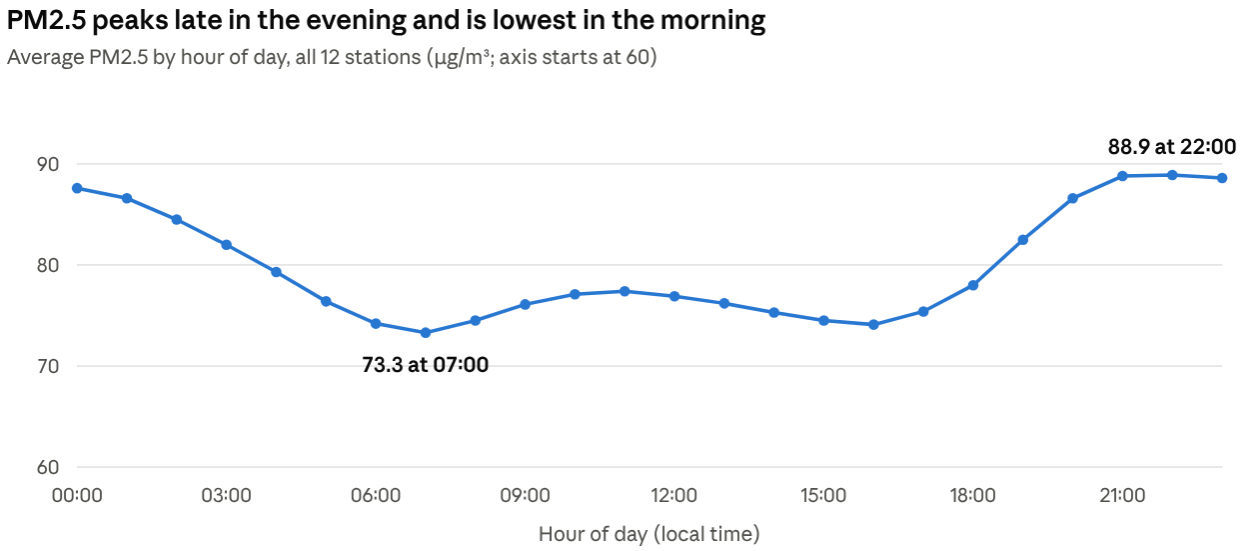
Correlation of next-hour PM2.5 with a trailing moving average of each window, Dongsi station



Computed from PRSA_Data_Dongsi · PM2.5.rolling(window).mean() vs PM2.5.shift(-1)

Moving averages answer a different question from lag features. Smoothing removes noise: the standard deviation of daily-mean PM2.5 drops from 73.9 to 26.2 µg/m³ under a 30-day average, which exposes the seasonal cycle in Q1. But every extra hour of averaging blurs the most recent reading, so next-hour correlation falls from 0.97 (current value) to 0.72 (24 h) and 0.29 (7 days). For short-term forecasting, a moving average is useful as trend context alongside the latest value, not as a replacement for it.

Q6. Is there a daily cycle?



Computed from the 12 station CSVs · mean PM2.5 grouped by the hour column

Yes. PM2.5 climbs through the evening to a peak of 88.9 µg/m³ at 22:00, stays high overnight, and bottoms out at 73.3 at 07:00, about 21% lower. The evening rise matches rush-hour traffic, heating and a shallow night-time boundary layer that traps particles; daytime mixing disperses them. This cycle is why `hour` is a model feature.

4. Task 1C: Model training and experiments

The tuned Random Forest predicts next-hour PM2.5 at Dongsi with MAE 11.15 µg/m³ and R^2 0.945 on an unseen final 20% of the timeline.

Model choice. We used a scikit-learn `Pipeline` of a median `SimpleImputer` followed by a `RandomForestRegressor`. A Random Forest captures non-linear effects (such as wind dispersing pollution only above some speed) without feature scaling, tolerates the skewed inputs, and reports feature importances we can interpret. Keeping the imputer inside the pipeline means the saved model carries its own preprocessing, which Task 4 relies on.

Features and target. Thirteen features describe the current hour: the 6 pollutants, 5 weather variables (`TEMP`, `PRES`, `DEWP`, `RAIN`, `WSPM`) and the calendar features `hour` and `month`. The target is PM2.5 shifted one hour forward.

Time-based split. Rows were kept in time order and split 80/20 with no shuffling, so the model is always tested on the future:

Set	Period	Hours
Training	2013-03-01 00:00 → 2016-05-05 16:00	27,450
Test	2016-05-05 17:00 → 2017-02-28 22:00	6,863

Hyperparameter tuning. `RandomizedSearchCV` sampled 6 of 27 combinations of `n_estimators` ∈ {50, 100, 150}, `max_depth` ∈ {10, 20, None} and `min_samples_leaf` ∈ {1, 2, 5}. It scored by MAE using `TimeSeriesSplit` (3 folds), so every validation fold lies after its training fold. To keep the search fast, it ran on the first 12,000 training hours; the winning settings were then refit on the full training set. The best settings were **150 trees, max depth 10, min 5 samples per leaf**: shallower, more regularised trees than the default.

Experiment table (test set, Dongsi):

Experiment	Settings	MAE (µg/m³)	RMSE (µg/m³)	R^2
Reference: persistence (next hour = this hour)	none	11.392	22.454	0.941
Experiment 1: Random Forest baseline	100 trees, unlimited depth, min leaf 1	11.558	22.164	0.942
Experiment 2: Tuned Random Forest	150 trees, depth 10, min leaf 5	**11.146**	**21.615**	**0.945**

Tuning cut MAE by 3.6% and RMSE by 2.5% against the baseline forest. The persistence row puts this in context: because PM2.5 is so persistent (Q4), simply repeating the current value is already a strong forecast. The untuned forest did not beat it on MAE, while the tuned forest beats it on all three metrics. It gains most on RMSE, meaning it handles large swings better.

Feature importance. Current PM2.5 accounts for 97.70% of the tuned model's importance, followed by PM10 (0.48%) and wind speed (0.44%). This matches the r = 0.97 lag-1 autocorrelation found in Q4. Importance shows what the model relies on, not cause and effect.

The tuned pipeline is saved as `models/best_rf.joblib`, and reloading it reproduces the Experiment 2 scores exactly.

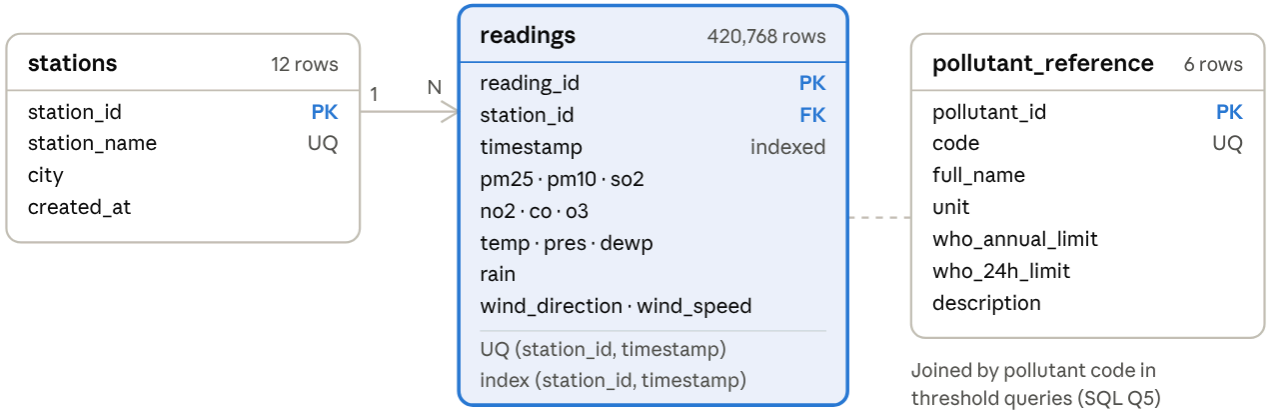
5. Task 2: Database design (MySQL and MongoDB)

The same 420,768 readings live in a normalised three-table MySQL schema and in one MongoDB collection of nested documents. Both are hosted (Aiven MySQL 8.4 and MongoDB Atlas) and indexed for the two queries a time series needs most: the latest reading and a date range.

5.1 Relational schema (MySQL)

Each station has many hourly readings; WHO limits sit in a lookup table

MySQL schema, database beijing_air_quality (hosted on Aiven as defaultdb)



PK primary key · FK foreign key (ON DELETE RESTRICT) · UQ unique · 1 to N: one station, many readings

Entity-relationship diagram · 3 tables, from database/schema_mysql.sql

Table	Role	Rows	Key constraints
`stations`	One row per monitoring station	12	PK `station_id`; `station_name` unique
`readings`	Hourly fact table: 6 pollutants, 6 weather values	420,768	PK `reading_id`; FK `station_id` → `stations`; unique (`station_id`, `timestamp`)
`pollutant_reference`	WHO 2021 guideline per pollutant	6	PK `pollutant_id`; `code` unique

Design choices:

- **Normalisation.** Station names move out of the fact table, so each reading stores a 4-byte id instead of repeating a name 35,064 times.
- **Time-series integrity.** The unique (`station_id` , `timestamp`) key makes a duplicate station-hour impossible; the API turns a violation into HTTP 409.
- **Indexes.** `idx_timestamp` and `idx_station_ts` (`station_id` , `timestamp`) serve date-range and latest-record queries without a full scan.
- **Reference data, not hard-coded limits.** Guideline values live in `pollutant_reference` , so threshold queries join to it (Query 5).
- **Safe deletes.** The foreign key uses `ON DELETE RESTRICT` , so a station with readings cannot be deleted.

The DDL is in `database/schema_mysql.sql` .

5.2 Document design (MongoDB)

Collection `air_quality_readings` holds one document per station-hour. Pollutants and weather sit in nested sub-documents, mirroring how the data is read; the station name is embedded instead of joined. Calendar parts are stored alongside the timestamp for cheap grouping. Sample document:

```
{
  "station": "Dongsi",
  "timestamp": "2017-02-28T23:00:00Z",
  "year": 2017, "month": 2, "day": 28, "hour": 23,
  "pollutants": { "pm25": 30.0, "pm10": 71.0, "so2": 11.0,
                  "no2": 87.0, "co": 1200.0, "o3": 4.0 },
  "weather": { "temp": 8.6, "pressure": 1014.1, "dew_point": -15.9,
               "rainfall": 0.0, "wind_direction": "NNE", "wind_speed": 1.3 }
}
```

Indexes: a unique compound index on (`station` , `timestamp`), a descending `timestamp` index for latest-record queries, and `pollutants.pm25` for threshold queries.

Aspect	MySQL `readings`	MongoDB `air_quality_readings`
Shape	Flat row, 15 columns	Nested `pollutants` / `weather` sub-documents
Station	Foreign key to `stations`	Embedded string
Schema enforcement	DDL types and constraints	Unique index + API validation

Aspect	MySQL `readings`	MongoDB `air_quality_readings`
Query style	SQL joins and `GROUP BY`	`find()` and aggregation pipelines

5.3 SQL queries and results

Q1. Latest reading per station. A subquery finds `MAX(timestamp)` per station and joins it back. All 12 stations return 2017-02-28 23:00, the final hour, e.g. Dongsi PM2.5 30.0, PM10 71.0, 8.6 °C, wind NNE 1.3 m/s.

Q2. Date range: Dongsi, 1–7 January 2016.

```
SELECT s.station_name, r.timestamp, r.pm25, r.pm10, r.temp, r.wind_direction
FROM readings r JOIN stations s ON r.station_id = s.station_id
WHERE s.station_name = 'Dongsi'
  AND r.timestamp BETWEEN '2016-01-01 00:00:00' AND '2016-01-07 23:00:00'
ORDER BY r.timestamp;
```

Result: 168 rows (7 days × 24 hours, none missing). It starts in a heavy episode: PM2.5 200, 193, 219 µg/m³ in the first three hours of the year.

Q3. Average, maximum and missing PM2.5 per station (`AVG` , `MAX` , `SUM(CASE WHEN pm25 IS NULL …)` grouped by station). Dongsi has the highest mean (86.19); Dingling the lowest (65.99); missing PM2.5 ranges from 1.1% (Wanliu) to 2.7% (Huairou).

Q4. Monthly average PM2.5 (`DATE_FORMAT(timestamp, '%Y-%m')` , 48 months). The worst months were December 2015 (150.52), February 2014 (148.32) and December 2016 (129.19); the cleanest were February 2016 (42.82) and August 2015 (44.18).

Q5. WHO guideline exceedance (joins `readings` to `pollutant_reference` , so the 15 µg/m³ limit is not hard-coded):

Station	Valid hours	Hours > 15 µg/m³	Share (%)	Mean PM2.5
Guanyuan	34,448	28,695	83.3	82.93
Gucheng	34,418	28,573	83.0	83.85
Wanliu	34,682	28,691	82.7	83.37
Aotizhongxin	34,139	28,022	82.1	82.77
Tiantan	34,387	28,149	81.9	82.16
Wanshouxigong	34,368	28,138	81.9	85.02

Station	Valid hours	Hours > 15 µg/m³	Share (%)	Mean PM2.5
Dongsi	34,314	28,017	81.6	86.19
Nongzhanguan	34,436	28,078	81.5	84.84
Shunyi	34,151	27,134	79.5	79.49
Changping	34,290	26,873	78.4	71.10
Huairou	34,111	26,445	77.5	69.63
Dingling	34,285	24,822	72.4	65.99

Every station exceeds the 24-hour guideline in at least 72% of hours, and every station mean is over 13 times the annual guideline of 5 µg/m³.

5.4 MongoDB queries and results

MQ1. Latest reading for a station: `find({station: "Dongsi"}).sort({timestamp: -1}).limit(1)` returns the 2017-02-28 23:00 document, identical to SQL Q1.

MQ2. Date range: `find({station: "Dongsi", timestamp: {$gte: ISODate("2016-01-01T00:00"), $lte: ISODate("2016-01-07T23:00")}}).sort({timestamp: 1})` returns 168 documents, matching SQL Q2 row for row.

MQ3. Average PM2.5 per station: a `$match` (non-null) → `$group` (`$avg`, `$max`, count) → `$sort` pipeline. The results equal SQL Q3 (e.g. Dongsi 86.19 over 34,314 documents), which cross-checks the two loads.

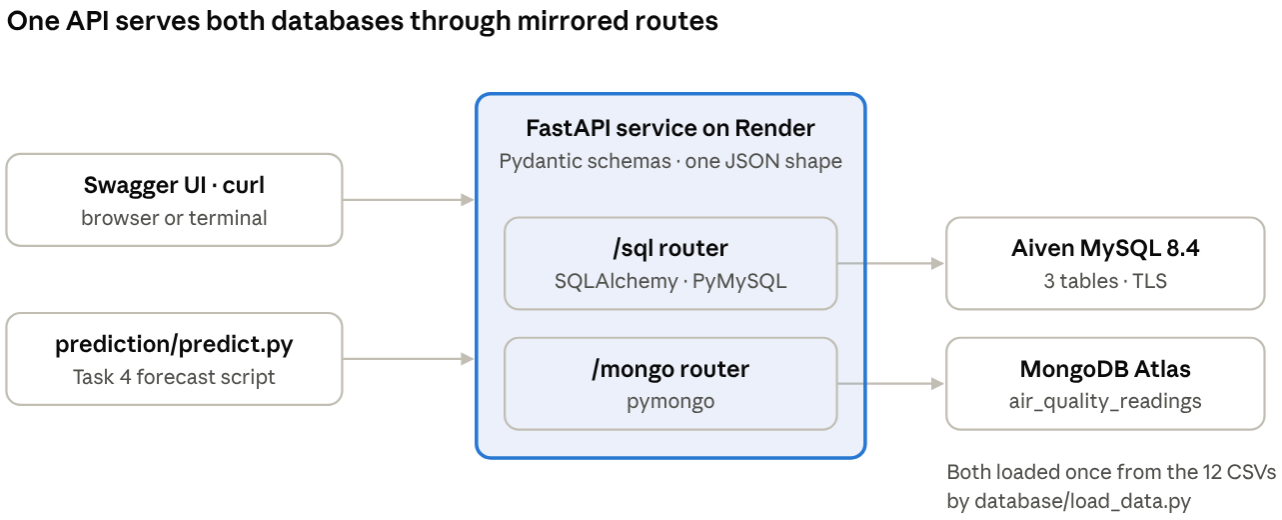
MQ4. Hazardous hours (PM2.5 > 150 µg/m³) per station:

```
db.air_quality_readings.aggregate([
  { $match: { "pollutants.pm25": { $ne: null } } },
  { $group: { _id: "$station", total: { $sum: 1 },
              hazardous: { $sum: { $cond: [{ $gt: ["$pollutants.pm25", 150]
}, 1, 0] } } } },
  { $addFields: { pct_hazardous: { $round: [{ $multiply: [{ $divide:
["$hazardous", "$total"] }, 100] }, 1] } } },
  { $sort: { pct_hazardous: -1 } }
])
```

Dongsi has the most hazardous hours (5,940; 17.3%), then Nongzhanguan (16.8%) and Wanshouxigong (16.6%). The suburban stations have the fewest: Changping 12.8%, Huairou 12.0%, Dingling 11.6% (3,984 hours).

6. Task 3: CRUD and time-series API

A single FastAPI service gives full CRUD plus latest-record and date-range queries on both databases. The two backends use mirrored routes (`/sql/…` and `/mongo/…`) and one shared JSON shape. The service is live at formative1mlpipeline.onrender.com/docs.



System architecture · 2 clients, 1 API, 2 hosted databases

Endpoints (`{db}` is `sql` or `mongo`):

Method	Path	Purpose	Responses
`POST`	`/{db}/readings`	Create a reading	201 created; 409 if the station-hour exists; 422 invalid body
`GET`	`/{db}/readings?station=&limit=&offset=`	List readings, newest first, paginated	200
`GET`	`/{db}/readings/{id}`	Read one reading	200; 404
`PUT`	`/{db}/readings/{id}`	Partial update: only the fields sent change	200; 404; 409
`DELETE`	`/{db}/readings/{id}`	Delete a reading	200; 404
`GET`	`/{db}/readings/latest?station=`	**Latest record**, overall or per station	200; 404 if empty

Method	Path	Purpose	Responses
`GET`	`/{db}/readings/range?start=&end=&station=&limit=`	**Records by date range**, inclusive, oldest first	200; 400 if `start` > `end`
`GET`	`/{db}/stations`	List stations	200

Implementation.

- **Structure.** `api/main.py` mounts two routers, `routers/sql_readings.py` and `routers/mongo_readings.py`. Data access is separated into `sql_db.py` (SQLAlchemy models of Nicole's three tables) and `mongo_db.py` (pymongo).
- **One contract for both databases.** Pydantic models in `schemas.py` validate every request: non-negative pollutant values, a station name of at most 50 characters, ISO-8601 timestamps. MongoDB stores Nicole's nested document; the API flattens it on the way out, so a client (Task 4) can switch databases by changing only the URL prefix.
- **Time-series queries use the indexes.** "Latest" is `ORDER BY timestamp DESC LIMIT 1` in SQL and `sort({timestamp: -1}).limit(1)` in MongoDB. "Range" is a `BETWEEN` / `$gte` – `$lte` filter. Both use the (station, timestamp) indexes, and results are capped at 5,000 rows to protect the service.
- **Configuration.** Connection strings come from environment variables (`.env`, never committed). Hosted MySQL connects over TLS (`SQL_SSL=true`).
- **Loading.** `database/load_data.py` reads the 12 CSVs and bulk-inserts them into either database in 10,000-row batches. Loading took about 90 s into Aiven and 280 s into Atlas.

Example (live):

```
curl "https://formative1mlpipeline.onrender.com/sql/readings/latest?
station=Dongsi"
```

```
{"id": "140256", "station": "Dongsi", "timestamp": "2017-02-28T23:00:00",
 "pm25": 30.0, "pm10": 71.0, "so2": 11.0, "no2": 87.0, "co": 1200.0, "o3":
4.0,
 "temp": 8.6, "pres": 1014.1, "dewp": -15.9, "rain": 0.0,
 "wind_direction": "NNE", "wind_speed": 1.3}
```

Testing. `tests/test_api.py` runs 26 tests: each of 12 behaviours is run against both backends, plus 2 MongoDB-only checks. They cover create and read, duplicate rejection (409), validation (422), list filtering, partial update, update into a duplicate (409), delete then 404, latest overall and per station, latest on an empty store (404), date ranges with station and limit, reversed dates (400), invalid ObjectId (400), and that MongoDB stores

the nested Task 2 layout. Tests use in-memory SQLite and `mongomock`, so they run in under a second and never touch the hosted data. We also exercised every endpoint against the hosted Aiven and Atlas databases: the Dongsi 1–7 January 2016 range returns 168 rows from each, matching Nicole's queries.

Hosting. The API runs on Render's free tier (`uvicorn api.main:app --host 0.0.0.0 --port $PORT`). The service sleeps after about 15 idle minutes, so the first request after a pause can take up to a minute.

7. Task 4: Prediction script

`prediction/predict.py` ties the pipeline together: it fetches the latest Dongsi record from the API, rebuilds Task 1's feature vector, loads the saved model and prints a next-hour forecast of 40.68 µg/m³.

The script works in four steps, matching the assignment brief:

1. **Fetch.** `fetch_latest_reading()` calls `GET /sql/readings/latest?station=Dongsi` with `requests` and raises on any HTTP error (`raise_for_status`). The API base URL is a single constant, so the same script runs against a local server or the live Render URL.
2. **Preprocess.** `prepare_features()` applies the Task 1 preprocessing to the API record. It maps the API field names back to the training column names (`pm25` → `PM2.5`, `wind_speed` → `WSPM`, and so on) and derives `hour` and `month` from the timestamp. It returns the 13 columns in exactly the training order.
3. **Load the model.** `joblib.load("models/best_rf.joblib")` restores the full tuned pipeline. Because the median imputer is part of that pipeline, a missing value in the fetched record is filled exactly as during training.
4. **Predict.** `predict_next_hour()` returns the forecast as a float, and `main()` prints it with the input it used.

Output (run against the live API):

```
Station: Dongsi
Latest recorded timestamp: 2017-02-28T23:00:00
Latest recorded PM2.5: 30.0
Predicted next-hour PM2.5: 40.68
```

The same record fetched through `/mongo/readings/latest` gives the identical prediction (40.68), confirming the two backends return the same data in the same shape. The dataset ends in February 2017, so this forecasts the hour after the last recorded one (2017-03-01 00:00), not today's air quality.

Testing. `tests/test_prediction.py` adds 5 tests: feature names, order and timestamp-derived values; the prediction call with a stand-in model; a reading with a missing field; an invalid timestamp; and an HTTP 500 from the API. With the 26 API tests, the full suite runs 31 tests, and all pass.

8. Repository, reproducibility and conclusion

Repository layout (GitHub):

Folder	Contents
`notebooks/`	Task 1 analysis and model notebook; Task 2 database notebook
`database/`	`schema_mysql.sql` (DDL), `load_data.py` (CSV → MySQL and MongoDB)
`api/`	FastAPI app, routers, schemas, database access
`prediction/`	`predict.py`
`models/`	`best_rf.joblib`, the tuned pipeline
`tests/`	26 API tests and 5 prediction tests

Raw data (`data/raw/`) and credentials (`.env`) are excluded from Git; `.env.example` lists the settings to provide.

Reproducing the results:

1. `pip install -r requirements.txt`, then copy `.env.example` to `.env` and fill in the database connection strings.
2. Download the Kaggle archive into `data/raw/` and run `python -m database.load_data` to load both databases.
3. Run `uvicorn api.main:app` and open `/docs`, or use the live URL.
4. Run `python prediction/predict.py` for a forecast and `pytest` for the 31 tests.

Limitations.

- **One station, one horizon.** The model is trained and evaluated on Dongsi for one hour ahead; we have not tested it on the other 11 stations or longer horizons.
- **Small gain over persistence.** The tuned forest beats "repeat the current value" by only 0.25 µg/m³ MAE. Lagged values and moving averages (Q4 and Q5) were analysed but not yet used as model features. Adding them, or trying a sequence model such as an LSTM, is the clearest next step.

- **Historical data.** The data ends on 2017-02-28, so the pipeline demonstrates forecasting on the latest stored record rather than live air quality.
- **Free hosting.** Render's free tier sleeps when idle, so the first API call can take up to a minute. Clients should allow a long timeout.

Conclusion. The four tasks fit together into a working pipeline. The exploration showed a highly persistent, strongly seasonal and diurnal pollutant that exceeds WHO guidelines at every station. Both databases store the full four-year history and answer the same queries with identical results. One API serves both, with CRUD and time-series endpoints, and the prediction script uses it to turn a live record into a next-hour forecast. All 31 automated tests pass.